

Functional Programming

Lab 01: The Countdown

V. Markos
Mediterranean College

Spring, 2024 – 2025, Week 14

Abstract

This lab aims to help us explore the really interesting problem “The Countdown” poses to us: how to reach a certain positive integer using common arithmetic and a finite number of other positive integers. We will first use brute-force to solve the problem and then optimise as much as possible.

1 The Problem

The Countdown problem (also, “the problem”, from now on) is a really simple one:

- Given a positive integer, t ;
- and a list of typically six positive integers, ℓ ;
- you have to use each $x \in \ell$ at most once and any of the four common arithmetic operations, $+$, $-$, $\times$, $\div$ to reach t .

In the corresponding TV show from which this problem stems there are also time restrictions (solve 10 instances of the game in 30 seconds, i.e., 3 seconds each), which we softly interpret as a challenge to build a Haskell script that solves this problem as fast as possible, hopefully within the realm of milliseconds.

2 Brute Force

The problem is essentially a search problem, since, from a vast search space of math expressions involving the given integers, we need to find those (if any) that reach the target value. Even if the search space seems (and is) vast, brute force seems the only reasonable approach at the moment¹. The following exercises

¹One might propose that a backwards DFS-like search where we start by applying each operation on the given target once and then recurse from that point downwards might also

will guide you through a simple implementation of a brute force solution using Haskell for the Countdown problem.

As a first step, let us sketch at an abstract level our approach:

1. First generate all possible rearrangements of the given numbers, containing at least one of those numbers.
2. Then, using those rearrangements, generate all possible expressions by properly inserting all available operators in all possible orders.
3. Then, evaluate all those expressions and compare their result with the target number, t , keeping only those that match it.

So, let us first warm up by defining some utilities:

Exercise 2.1. Start by defining some useful data types that we are going to use throughout this lab. Namely, define:

- A data type to capture the four mathematical operations we are allowed to use: addition, subtraction, multiplication and division.
- A data type to capture mathematical expressions. For that purpose, draw inspiration from your last semester assessment and try to be as minimalistic as possible.

In both cases, derive from any typeclasses you need to (e.g., `Show` to make your things printable).

Exercise 2.2. Implement some utility functions that we will need throughout all our approaches, namely:

- A function `valid` that takes an operator and two integers and returns a boolean value depending on whether the given operation on the two integers at hand is valid or not, where validity is to be interpreted as whether the result is a positive integer.
- A function `apply` that takes an operator and two integers and returns the integer that is the result of applying that operator to the two given integer arguments.
- A function `nesplit` that takes a list and returns a list of tuples containing all the possible *non-empty* splits of that list in two parts.
- A function `subbags` that takes a list and returns a list of all possible permutations of all sublists of that list.

For the above you might find useful to use `subsequences` and `permutations` from `Data.List` or implement variants of those on your own.

be an option. This is actually true and, if we want to find *just one* solution, it suffices. If we want to find all solutions, though, we would end up exploring the entire search space, which is just as applying brute force.

Having all necessary tools, we might now proceed to implement some actual solution to our problem.

Exercise 2.3. Define a function `exprs` that takes a list of integers and computes a list of all mathematically valid expressions within our context.

Hint: To that end you might also want to define a helper function, `combine`, which takes two expressions and returns the list of all valid combinations of those expressions.

Exercise 2.4. Having generated all possible expressions from ℓ , we have yet to define a function that evaluates an expression to its integer value. Define a function, `eval`, that performs this task, i.e., `eval` should take a single expression and return a list of values containing the value of that expression.

Hint: Generating a singleton list might seem a bit absurd, however this might help you handle the recursive generation of all possible intermediate results. In case you devise another way to do so that does not require lists, feel free to do so!

Exercise 2.5. Eventually, define a function `solutions` that takes a list of integers and a single target integer and returns the list of all valid mathematical expressions that reach this target integer.

3 Getting Efficient

Having implemented a function that can actually compute all possible solutions for an instance of the Countdown problem, how about some optimisation?

Exercise 3.1. Optimising a brute force approach that actually works often boils down to efficiently pruning down the search space of the problem. To that end, start by considering whether we can narrow down the number of expressions we generate in each case. For instance, given $\ell = [2, 5]$ and a single operation, $+$, is there any difference between $2 + 5$ and $5 + 2$? If not, why compute both?

Think for other possible optimisations in the same spirit and implement them by appropriately modifying your Haskell script. Measure any performance gains using `ghci` and `:set +s`.

Exercise 3.2. Observe how the approach we adopted above performs some needless computations. Namely, given that many expressions involving subtraction and division will eventually end up to be invalid, since they yield non-positive results, we should better prune such expressions as soon as possible. So, instead of first generating and then evaluate all possible expressions, why not merge those two processes into one?

Modify `solutions` appropriately so as to implement the above idea of computing the value of an expression once it is being generated. Make any other modifications required and define any auxiliary data types you might need to.

Exercise 3.3 (Optional). As a side effect² of the above, we have ended up with a solution that computes all valid expressions that reach the target number, t , up to any restrictions we have imposed in Exercise 3.1. So, e.g., from the two expressions from $\ell = [3, 7]$ that can reach 10 using $+$, $3 + 7$ and $7 + 3$, we are examining and computing just one, say $3 + 7$. While not false, such a solution list is incomplete in a strict sense of generating *all* such expressions. Define a function, `expand`, that gets a list of expressions and generates all expressions pruned by your implementation of Exercise 3.1.

²In Functional Programming we do not like side effects in any reasonable sense...